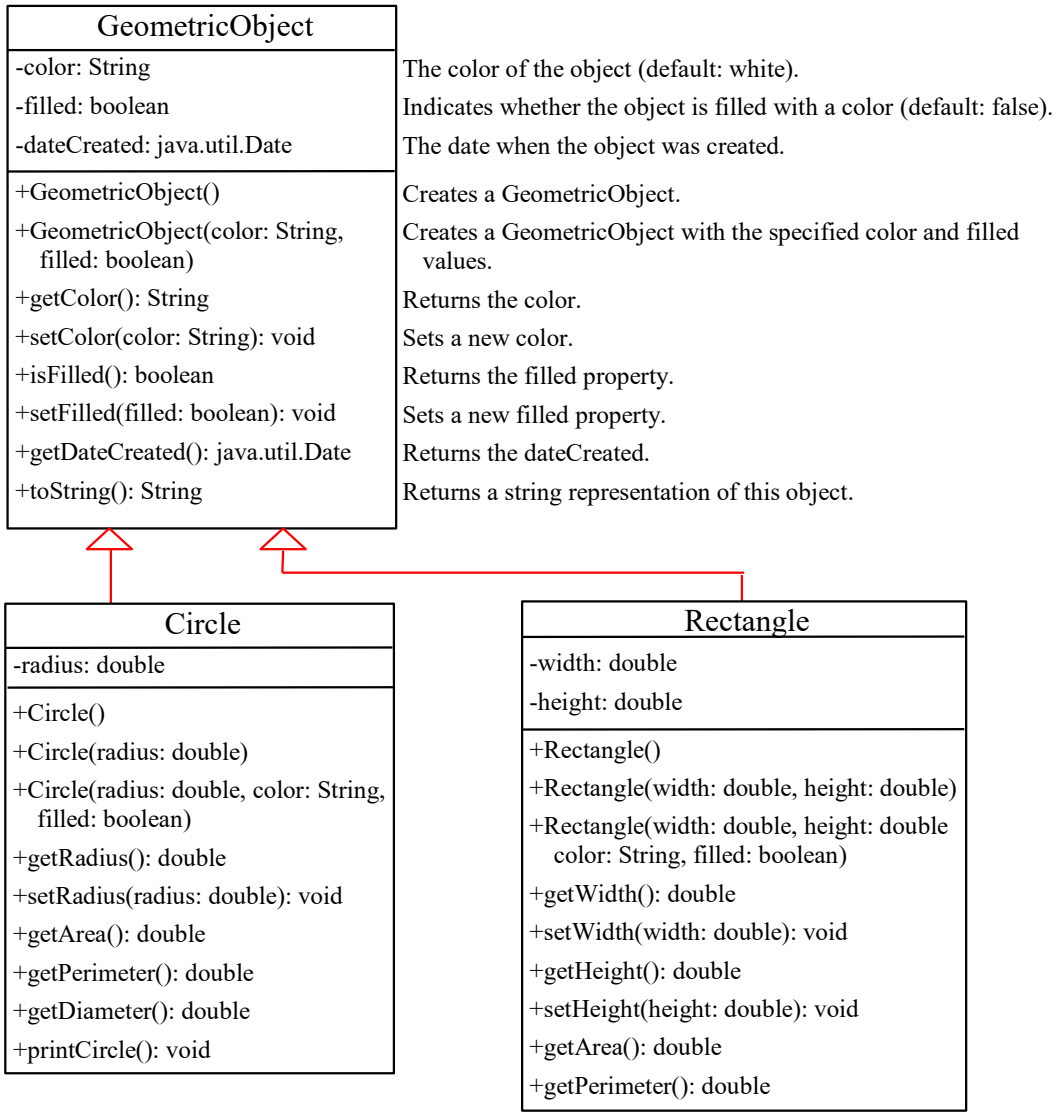
Object Oriented Programming

Mihai Dascălu

Inheritance and Polymorphism

Superclasses and Subclasses



TestGeometricObject

Are superclass's Constructor Inherited?

- No, they are not inherited
- Invoked **explicitly** (using the super keyword) or **implicitly**
- A constructor is used to construct an instance of a class
- Unlike properties and methods, a superclass's constructors are not inherited in the subclass
- Superclass's constructor can only be invoked from the subclasses' constructors, using the keyword super
- *If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked*

Superclass's Constructor Is Always Invoked

- A constructor may invoke an overloaded constructor or its superclass's constructor
- If none of them is invoked explicitly, the compiler puts super() as the first statement in the constructor

```
public A() {
}
```

is equivalent to

```
public A() {
  super();
}
```

```
public A(double d) {
  // some statements
}
```

is equivalent to

```
public A(double d) {
  super();
  // some statements
}
```

Using the Keyword `super`

- The keyword super refers to the superclass of the class in which `super` appears
- Can be used in two ways:
  - To call a superclass constructor
  - To call a superclass method
  - Access a superclass's attribute that may be shadowed
- CAUTION
  - You must use the keyword super to call the superclass constructor

Balatagul & Morometii

Do or Do not

- You can use **either** this(…) or super(…) as a first instruction, but **not both** in the constructor
- this(…) or super (…) need to be the **first instructions**
- Two classes cannot be each other's superclass <-> cyclic inheritance
- A class can extend one and only one other class

Constructor Chaining

- Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain

Faculty

Example on the Impact of a Superclass without no-arg Constructor

```
public class Apple extends Fruit {
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor is invoked");
  }
}
```

Errors? What errors?

Declaring a Subclass

- A subclass extends properties and methods from the superclass:
  - Add new properties
  - Add new methods
  - Override the methods of the superclass
- e.g., rewriting the printCircle() method in the Circle class as follows & calling superclass methods:

```
public void printCircle() {
  System.out.println("The circle is created " +
    super.getDateCreated() + " and the radius is "
+ radius);
}
```

Overriding Methods in the Superclass

- A subclass inherits methods from a superclass
- *Method overriding* = modify the implementation of a method defined in the superclass
  - The return type of the overriding method **must match exactly** to the return type of the overridden method if the return type is a *primitive*
  - In case of objects, the return type of the overriding method may be a *subclass* of the return type of the overridden method

```
public class Circle extends GeometricObject {
  // Other methods are omitted
  /** Override the toString method */
  public String toString() {
    return super.toString() + "\nradius is " + radius;
  }
}
```

Note

- An instance method can be overridden only if it is accessible
  - A **private method cannot be overridden**, because it is not accessible outside its own class
- If a **method** defined in a **subclass** is **private** in its **superclass**, the two *methods are completely unrelated*
- Like an instance method, a **static** method **can be inherited**
  - A static method **cannot** be overridden
  - If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is **hidden**
- Variables are **shadowed** and methods are **overridden**
- A class cannot override the super class's constructor
  - Constructors are not inherited

Overriding vs Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

The Object Class and Its Methods

- Every class in Java is descended from the java.lang.Object class
- If no inheritance is specified when a class is defined, the superclass of the class is Object

```
public class Circle {
   ...
}
```

Equivalent

```
public class Circle extends Object {
   ...
}
```

The toString() method in Object

- The toString() method returns a string representation of the object
- The default implementation returns a string consisting of a class name + (@) + a number representing this object

```
A a= new A();
System.out.println(a.toString());
```

a@15037e5 – not very helpful or informative

- Usually you should override the toString method so it returns a digestible string representation of the object

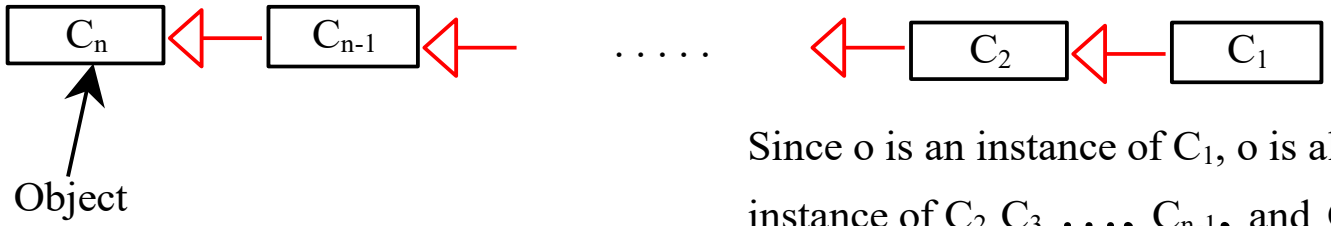
Polymorphism, Dynamic Binding and Generic Programming

- An object of a subtype can be used wherever its supertype value is required => polymorphism
- Method *m*() takes a parameter *x* of the Object type
  - Upon execution, the toString() of *x* is invoked
  - *x* may be an instance of any Object
- The used implementation is determined dynamically by the Java Virtual Machine at runtime => dynamic binding

VariousStudents

Dynamic Binding

- Suppose an object o is an instance of classes C_1, C_2, ..., C_{n-1}, and C_n, where
  - C_1 is a subclass of C_2
  - C_2 is a subclass of C_3, ...
  - C_{n-1} is a subclass of C_n
- C_n is the most general class, and C_1 is the most specific class
- In Java, C_n is the Object class
- If o invokes a method p, the JVM searches the implementation for the method p in C_1, C_2, ..., C_{n-1} and C_n, in this order, until it is found
- Once an implementation is found, the search stops and the **first-found implementation** is invoked



Since o is an instance of C_1, o is also an instance of C_2, C_3, . . ., C_{n-1}, and C_n

Method Matching versus Binding

- Matching a method signature and binding a method implementation are two issues
- The compiler finds a **matching method** according to parameter type, number of parameters, and order of the parameters at **compilation** time
- A method may be implemented in several subclasses
  - The Java Virtual Machine **dynamically binds** the implementation of the method at **runtime**

Generic Programming

- Polymorphism allows methods to be used generically for a wide range of object arguments => *generic programming*
- If a method's parameter type is a superclass, you may pass an object to this method
- When an object is used, the object's implementation is determined dynamically & invoked - e.g., toString()

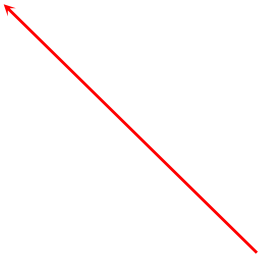
Casting Objects

- Casting can also be used to convert an object of one class type to another within an inheritance hierarchy

```
m(new Student());
```

- Assigns the object new Student() to a parameter of the Object type. This statement is equivalent to:

```
Object o = new Student(); // Implicit casting
m(o);
```



The statement Object o = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object

Why Casting Is Necessary?

- Suppose you want to assign the object reference o to a variable of the Student type:

  `Student b = o;` => compilation error occurs

  - A Student object is always an instance of Object, but an Object is not necessarily an instance of Student

- Need to use an explicit casting

- Syntax like the one used to cast primitive data types:

```
Student b = (Student)o; // Explicit casting
```

Casting from Superclass to Subclass

- Explicit casting must be used when casting an object from a superclass to a subclass
- This type of casting may not always succeed

```
GraduateStudent x = (GraduateStudent)stud;

Student x = (Student)person;
```

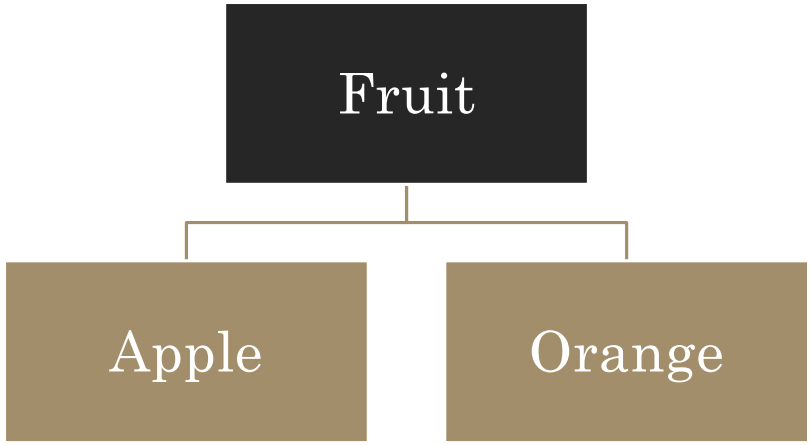
The `instanceof` Operator

- Use `instanceof` operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();
... // Some lines of code
/** Perform casting if myObject is an instance of
  Circle */
if (myObject instanceof Circle) {
   System.out.println("The circle diameter is"+
     ((Circle)myObject).getDiameter());
   ...
}
```

Tip

- You may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange
- An apple is a fruit, so you can always safely assign an instance of Apple to a variable for Fruit
- However, a fruit is not necessarily an apple, so you must use explicit casting to assign an instance of Fruit to a variable of Apple



The `equals` Method

- Compares the contents of two objects
- Default implementation in the Object class :

```
public boolean equals(Object obj) {
  return (this == obj);
}
```

For example, the equals method is overridden in the Circle class

```
public boolean equals(Object o) {
  if (o instanceof Circle) {
    return radius == ((Circle)o).radius;
  }
  else
    return false;
}
```

Note

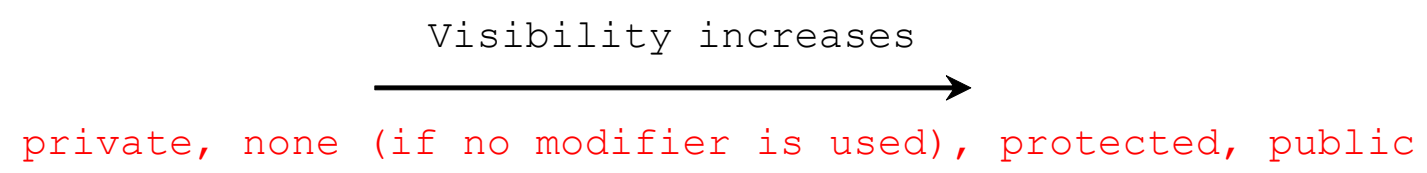
- The == comparison operator is used for:
  - Comparing two primitive **data type values**

  or
  - Determining whether two objects have the same reference
- The equals method is intended to test whether two objects have the **same contents**, provided that the method is modified in the defining class of the objects
- The == operator is stronger than the equals method

The `protected` Modifier

- The `protected` modifier can be applied on **data** and **methods** in a class

- A protected data or a protected method in a public class can be **accessed by any class** in the **same package** or its **subclasses**, even *if the subclasses are in a different package*



Using Visibility Modifiers (1)

- At the top level (class): public or default
- At the member level: public, protected, default, or private
- Use the most restrictive access level that makes sense for a member
- Use private unless you have a good reason not to

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

Using Visibility Modifiers (2)

- Avoid public fields except for constants
  - Public fields tend to link you to an implementation and limit your flexibility in changing your code
- Each class can present two contracts
  - One for the users of the class
  - One for the extenders of the class
- Make the *fields private* and *accessor* methods *public* if they are *intended for the users* of the class
- Make the fields or method *protected* if they are *intended for extenders* of the class
- The contract for the extenders *encompasses* the contract for the users

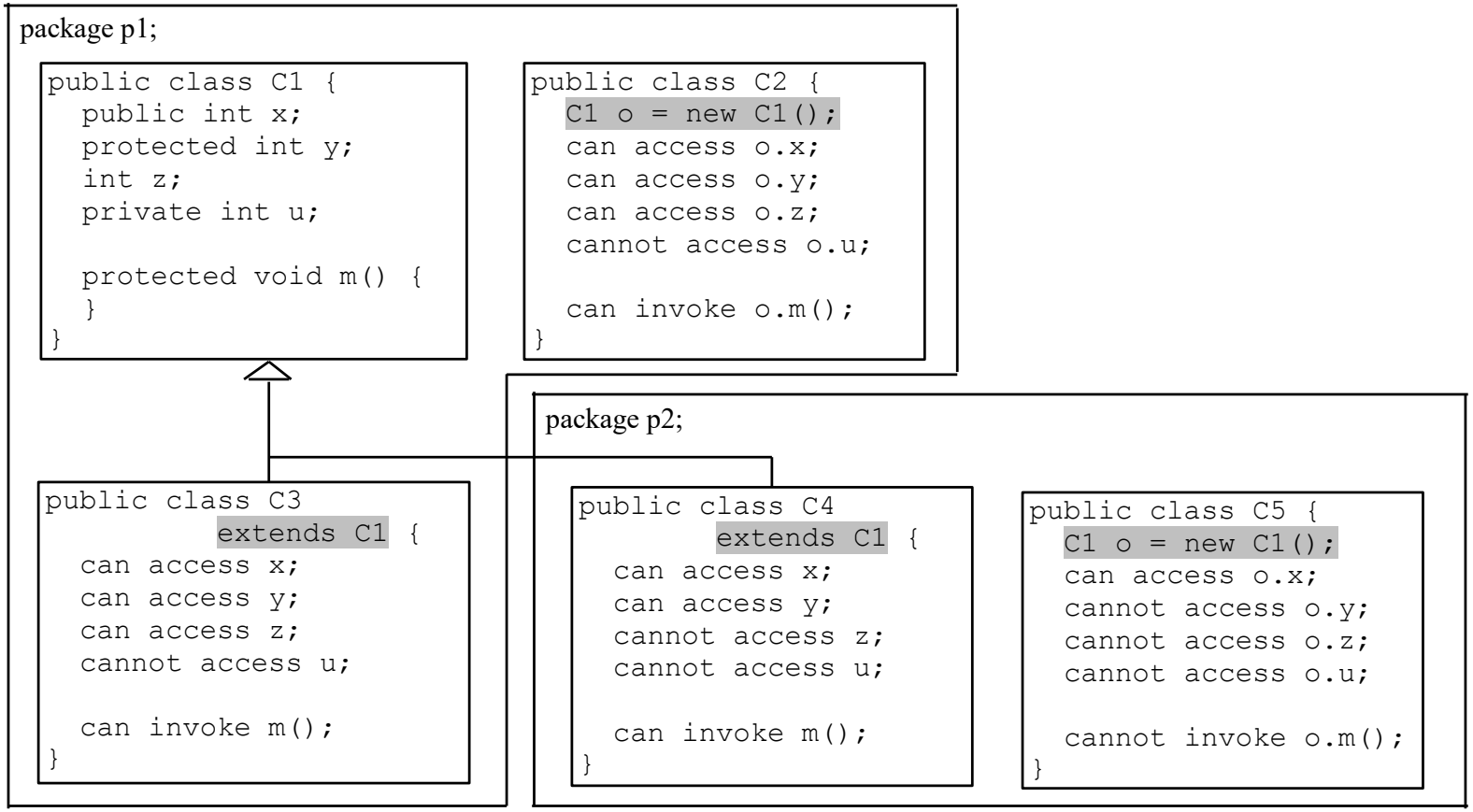
Using Visibility Modifiers (3)

- The extended class may increase the visibility of an instance method from protected to public, or change its implementation
  - You should never change the implementation in a way that violates that contract
- A subclass may override a protected method in its superclass and change its visibility to public
  - A subclass **cannot weaken the accessibility** of a method defined in the superclass
  - For example, if a method is defined as public in the superclass, it must be defined as public in the subclass

Using Visibility Modifiers (4)

- A class should use the private modifier to hide its data from direct access by clients
  - You can use *get* and *set* methods to provide access to the private data, but only to data you want the user to see or to modify
  - A class should also hide methods not intended for client use
- A property *shared by all* the instances of the class should be declared as static

Example Visibility Modifiers



The `final` Modifier

- The modifiers are used on classes and class members (data and methods), **even on local variables** in a method
- The `final` class cannot be extended (e.g., most JDK classes):

```
final class Math {

  ...

}
```

- The `final` variable is a constant:

```
final static double PI = 3.14159;
```

- A final local variable is a constant inside a method
- The `final` method cannot be overridden by its subclasses

What about static contexts?

- If a subclass defines a static method with the same signature as a static method in the superclass, then
  - The method in the subclass hides the one in the superclass
  - Static methods in interfaces are never inherited!
- Static elements are initialized all the way in the inheritance chain
- A static initializer declared in a class is executed **when the class is initialized**
- Static initializers may be used to initialize the class variables of the class together with any field initializers for class variables
- A static initialization block is automatically called when its class loads (i.e., constructor invoked, retrieve static field)

StaticTest